0 1 0 0 0 0 0 1

'A' = 65 = 01000001

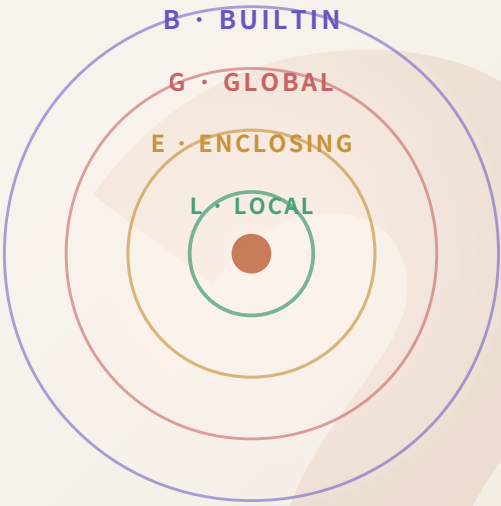
HONG

PYTHON · 공식 문서 · 4단계 학습

PyWhy

공식 문서로 분해하는 파이썬 사고법

컴퓨터는 어떻게 생각하는가



초급 중급 고급 전공자

PYWHY

머리말

"왜 이렇게 동작하는가"를 외우기 전에 묻는 책

이 책이 다른 파이썬 책과 다른 점

파이썬 입문서 대부분은 **"무엇을 쓰는가"**로 시작합니다. `for`를 쓴다, `def`로 함수를 만든다, `class`로 객체를 정의한다. 그리고 문법을 외우게 합니다.

이 책은 다릅니다. 매 챕터마다 묻는 질문은 하나입니다 — **"왜 이렇게 동작하는가."**

> - `print()`는 왜 명령어가 아니라 함수인가?
> - `for x in lst:` 한 줄 뒤에 어떤 분해가 숨어있는가?
> - `x = 1`은 왜 "상자에 넣기"가 아니라 "이름표 붙이기"인가?
> - 왜 리스트는 `append`되는데 튜플은 안 되는가?
> - `super()`는 왜 "부모"가 아니라 "MRO의 다음 클래스"인가?
> - `await`는 정확히 무엇을 기다리는가?

AI 시대에 코드를 외우는 것은 무의미합니다. AI가 항상 더 빨리 적어줍니다. 하지만 **AI가 만든 코드를 검증할 수 있는 사람**은 인터프리터의 사고 방식을 이해한 사람뿐입니다. 이 책은 그것을 기르는 책입니다.

이 책을 읽는 법 — 4단계 시스템

매 챕터는 같은 질문을 4번 다른 깊이로 답합니다. 처음에는 자기 수준에 맞는 단계만 읽으세요. 나중에 같은 챕터를 다시 펼치면, 다음 단계를 읽을 준비가 되어 있을 것입니다.

단계	대상	다루는 깊이	독자에게 남기는 것
초급	처음 코딩하는 학생	실생활 비유, 기본 코드 실행	"이 코드는 이렇게 쓴다"
중급	고등학생 / 대학 비전공	왜 이렇게 동작하는지, 흔한 함정	"이 코드는 왜 이렇게 동작한다"
고급	대학 입문 ~ 전공 1–2년	원서 용어, 내부 메커니즘	"이 메커니즘은 어떤 원리에서 나온다"
전공자	CS 전공 + 현직 백엔드	CPython 바이트코드, C 구조체, PEP·논문 인용	"이 메커니즘은 어떻게 구현되어 있다"

독자별 권장 학습 경로

첫 코딩 — 1–9주차의 초급만 차례대로. 모든 코드 직접 실행. 중급은 의문이 생긴 부분만.

비전공 백엔드 전환자 — 1–15주차의 초급 + 중급. 고급은 자기 업무에 직결된 주차(7 for, 12 클래스, 14 타입 힌트, 15 asyncio)부터.

CS 전공 학부생 — 1–15주차의 중급 + 고급. 전공자 영역은 CPython 소스를 옆에 두고. PEP 인용은 직접 peps.python.org에서 확인.

현직 + AI 시대 적응 — 각 주차의 § AI 섹션부터 거꾸로. AI가 만든 코드의 함정 사례 → 원리 이해 → 다른 단계로 깊이 이동.

인용·검증 정책

이 책의 모든 기술적 주장은 다음 1차 출처와 직접 대조해 검증했습니다:

1. **Python 3.14 공식 문서** — `docs.python.org/3` (한국어판: `docs.python.org/ko/3`)
2. **PEP 문서** — `peps.python.org`
3. **CPython 소스 코드** — `github.com/python/cpython`
4. **국제 표준 문서** — IEEE 754, Unicode, ISO/IEC 9899 등

각 챕터 말미에 참고문헌 섹션이 있습니다. 의심나는 주장은 그 출처를 직접 확인하세요. 이 책의 의도는 **독자가 이 책을 검증할 수 있게 만드는 것**입니다.

버전 기준

Python 3.14 (CPython)를 1차 기준으로 작성됐습니다. 바이트코드 출력 예시는 3.12–3.13 환경에서 캡처한 것이 일부 있고, 해당 부분은 별도 안내가 있습니다. 3.13에서 도입된 변경(예: PEP 667 frame fast/locals 단일화), 3.14에서 도입·제거된 변경(예: PEP 779 free-threading 정식 지원, `RETURN_CONST` 옵코드 제거)은 본문에 표시했습니다.

마지막으로

이 책의 모든 비유는 책의 끝까지 일관됩니다. 1주차에 본 "이름표"는 9주차 함수 인자에서 다시 등장하고, 12주차 던더 메서드에서 또 등장합니다. 비유를 한 번 잡으면 그 다음 챕터들이 자연스럽게 연결됩니다.

코드를 외우려 하지 말고, 인터프리터의 시선을 빌리세요.

H0NG

PyWhy v1.0

WEEK 01

컴퓨터, AI, 그리고 코딩

코딩을 배우기 전에, 먼저 **컴퓨터가 무엇인지**, **프로그래밍 언어가 왜 필요한지**, 그리고 **AI 시대에 코딩을 배우는 것이 왜 중요한지**를 이해합니다.

이번 주의 핵심 질문: "컴퓨터는 어떻게 생각하는가?"

≡ 목차 — 1주차 전체 내용

01 이번 주 이야기

02 왜 코딩을 배우는가?

03 컴퓨터란 무엇인가?
초급 중급 고급

04 프로그래밍 언어는 왜 있는가?
초급 중급 고급

05 왜 파이썬인가?
초급 중급 고급 전공자

06 print() — 첫 번째 함수
초급 중급 고급 전공자

07 AI는 코딩을 어떻게 하는가?
초급 중급 고급 전공자

08 컴퓨터는 print()를 어떻게 처리하나?
전공자

09 실습: 파이썬 첫 실행

10 REPL — 파이썬과 직접 대화하기
초급 중급 고급 전공자

11 연습문제 (10문제 — 7+3 전공자)
초급 중급 고급 전공자

12 이번 주 정리

13 용어 사전 (27개)

14 다음 주 맛보기

이번 주 이야기

여러분이 처음 보는 외국인을 만났다고 상상해보세요. 그 사람은 여러분의 말을 전혀 못 알아듣습니다. 여러분도 그 사람의 말을 전혀 못 알아듣습니다.

하지만 둘 다 **숫자**는 이해합니다. 손가락으로 3을 보여주면, 상대방도 3이라는 걸 압니다.

컴퓨터도 마찬가지입니다. 컴퓨터는 한국어도, 영어도 모릅니다. 오직 **0과 1**만 이해합니다. 그런데 우리는 컴퓨터에게 "안녕하세요"라고 출력하라고 시킬 수 있습니다. 이것이 어떻게 가능할까요?

그 비밀이 바로 **프로그래밍 언어**입니다. 그리고 그중에서도 가장 사람의 말에 가까운 언어가 **파이썬(Python)**입니다.

왜 이걸 배우는가?

WHY — 왜 코딩을 배워야 하는가?

"AI가 코드를 대신 짜주는 시대에 왜 코딩을 배워야 할까?"
이건 마치 "번역기가 있는데 왜 영어를 배워야 하나?"와 같은 질문입니다.

번역기는 말을 옮겨주지만, **무슨 말을 해야 하는지**는 알려주지 않습니다.
AI는 코드를 짜주지만, **무엇을 만들어야 하는지**, **이 코드가 맞는지**는 알려주지 않습니다.

코딩을 배우는 진짜 이유는 코드를 외우기 위해서가 아닙니다.
"컴퓨터가 어떻게 생각하는지"를 이해하기 위해서입니다.
이걸 이해하면, AI에게 정확한 지시를 내릴 수 있고, AI가 만든 결과를 판단할 수 있습니다.

컴퓨터란 무엇인가?

초급 **컴퓨터 = 아주 빠른 계산기**

컴퓨터의 정체는 생각보다 단순합니다. **아주아주 빠른 계산기**입니다.

여러분이 "1 + 1"을 계산하는 데 1초가 걸린다면,
컴퓨터는 같은 시간에 **수십억 번**의 계산을 합니다.
이것이 컴퓨터의 유일한 능력입니다: **엄청나게 빠른 계산**.

컴퓨터는 똑똑하지 않습니다. 창의적이지도 않습니다.
우리가 시킨 것을 **정확히, 빠르게, 지치지 않고** 반복할 뿐입니다.
"생각"하는 것처럼 보이는 건, 사실 엄청난 속도로 계산을 반복하는 것입니다.

중급

컴퓨터의 3가지 핵심 부품

컴퓨터는 크게 3가지 부품으로 구성됩니다:

부품	비유	역할
CPU (중앙처리장치)	사람의 **두뇌**	계산을 실행하는 곳. `1 + 1 = 2`를 처리
메모리 (RAM)	**책상**	지금 작업 중인 데이터를 올려두는 곳. 전원 끄면 사라짐
저장장치 (SSD/HDD)	**서랍장**	파일을 영구 보관하는 곳. 전원 꺼도 유지

프로그램이 실행되면 이런 과정이 벌어집니다:

1. 저장장치(서랍장)에서 프로그램 파일을 꺼냄
2. 메모리(책상)에 올려놓음
3. CPU(두뇌)가 한 줄씩 읽으면서 실행

프로그램 실행 = 서랍장에서 꺼내서 → 책상에 올리고 → 두뇌가 처리

고급 **컴퓨터는 왜 0과 1만 이해하는가?**

컴퓨터의 가장 기본 단위는 **트랜지스터(transistor)**입니다. 트랜지스터는 전기 스위치로, 딱 두 가지 상태만 가집니다:

- **전류가 흐른다** → 1 (ON)
- **전류가 안 흐른다** → 0 (OFF)

현대 CPU에는 이 트랜지스터가 **수십억 개** 들어있습니다.
이 0과 1의 조합만으로 모든 계산, 모든 문자, 모든 이미지를 표현합니다.

- 숫자 `42` → 이진수 `101010`
- 문자 `'A'` → ASCII 코드 `65` → 이진수 `1000001`
- 색상 빨강 → RGB `(255, 0, 0)` → 각각 이진수로 변환

즉, 컴퓨터가 하는 **모든 일**은 결국 0과 1의 조합을 바꾸는 것입니다.
프로그래밍 언어는 인간의 의도를 이 0과 1로 **번역**해주는 도구입니다.

※ `'A'` = 65 의 매핑은 ASCII / Unicode 표준에서 정의된 코드 포인트 `U+0041` 입니다[2].

Python 공식 문서에서는...

Python Tutorial 1장 "입맛 돋우기(Whetting Your Appetite)"는 파이썬을 이렇게 소개합니다:
"파이썬은 인터프리터 언어입니다. 컴파일과 링크 단계가 필요 없으므로 개발 시간을 상당히 단축해줍니다."[1]
즉, 파이썬은 우리가 쓴 코드를 한 줄씩 즉시 컴퓨터가 실행할 수 있는 명령으로 바꿔서 실행하는 언어입니다.

프로그래밍 언어란 무엇인가?

초급 **통역사가 필요하다**

우리가 컴퓨터에게 "안녕하세요를 화면에 보여줘"라고 말하면, 컴퓨터는 이해 못합니다.
컴퓨터는 `01001000 01100101 01101100` 같은 숫자만 이해하니까요.

그래서 **통역사**가 필요합니다.
이 통역사 역할을 하는 것이 바로 **프로그래밍 언어**입니다.

프로그래밍 언어에도 종류가 있습니다. 마치 한국어, 영어, 일본어처럼:

언어	특징	비유
기계어	0과 1로만 이루어짐	컴퓨터의 모국어
C 언어	기계에 가까운 언어	법률 문서 (정확하지만 어려움)
Java	범용 언어	표준어 (정확하고 격식있음)
Python	사람에게 가까운 언어	일상 대화 (쉽고 자연스러움)

중급

컴파일러 vs 인터프리터 — 통역 방식의 차이

프로그래밍 언어를 0과 1로 바꾸는 방법은 크게 두 가지입니다:

	컴파일러 (COMPILER)	인터프리터 (INTERPRETER)
비유	책 전체를 번역한 후 출판	동시통역사가 한 문장씩 바로 통역
과정	전체 코드 → 기계어 파일 → 실행	한 줄 읽기 → 즉시 실행 → 다음 줄
대표 언어	C, C++, Go, Rust	**Python**, JavaScript, Ruby
장점	실행 속도 빠름	개발 속도 빠름, 즉시 테스트 가능
단점	수정할 때마다 전체 재번역	실행 속도 상대적으로 느림

WHY — 왜 파이썬은 인터프리터 방식인가?

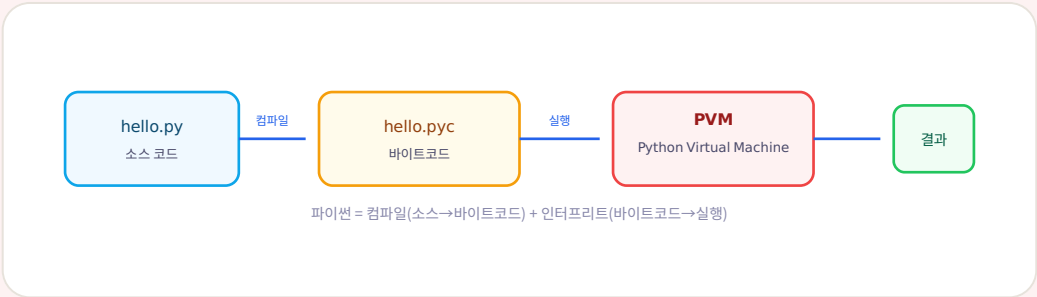
파이썬의 철학은 **"개발 속도 > 실행 속도"**입니다.
코드를 쓰자마자 바로 실행해볼 수 있어야 배우기 쉽고, 실험하기 쉽고, 빨리 만들 수 있습니다.
공식 문서에서도 이렇게 말합니다:
"인터프리터는 대화형으로 사용할 수 있어서, 언어의 기능을 실험하거나, 쓰고 버릴 프로그램을 만들거나, 바닥부터 프로그램을 만들어가는 동안 함수들을 테스트하기 쉽습니다."[1]

고급

실제로 파이썬이 코드를 실행하는 과정

CPython(우리가 보통 쓰는 파이썬 구현체)은 순수 인터프리터가 아닙니다. 실제로는 **2단계**를 거칩니다:

1. **컴파일 단계**: 소스 코드(.py)를 **바이트코드(Bytecode)**로 변환해서 `__pycache__/` 폴더에 `.pyc` 파일로 캐시[3]
2. **실행 단계**: 인터프리터(흔히 "PVM"이라 부름 — 비공식 용어)가 바이트코드를 한 명령(opcode)씩 실행



이것이 Java의 JVM과 비슷한 구조입니다. "인터프리터 언어"라고 불리지만, 내부에는 컴파일 과정이 숨어있습니다. 파이썬이 처음 실행할 때 `__pycache__` 폴더에 `module.cpython-3XX.pyc` 형식의 파일을 만드는 이유가 바로 이것입니다 (파일명에 인터프리터 버전 태그가 포함됨)[3].

왜 파이썬인가?

초급

가장 "사람 말"에 가까운 언어

같은 일을 시키는데, 언어마다 이렇게 다릅니다:

C 언어로 "Hello"를 출력하려면:

C 언어

```
#include <stdio.h>
int main() {
    printf("Hello");
```

```
    return 0;
}
```

Java로 "Hello"를 출력하려면:

```Java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

Python으로 "Hello"를 출력하려면:

```Python
print("Hello")
```

딱 **한 줄**입니다. 이것이 파이썬이 전 세계에서 가장 많이 배우는 첫 번째 프로그래밍 언어인 이유입니다.

중급

파이썬의 설계 철학

파이썬을 만든 **귀도 반 로섬(Guido van Rossum)**은 1989년 12월 크리스마스 휴가 때 파이썬을 만들기 시작했습니다[4]. 그의 핵심 철학:

> "아름다운 것이 추한 것보다 낫다. 명시적인 것이 암시적인 것보다 낫다. 단순한 것이 복잡한 것보다 낫다."
> — The Zen of Python (파이썬의 선) [5]

이 철학 때문에 파이썬에는 다른 언어에 없는 특징들이 있습니다:

- **들여쓰기가 문법이다**: 중괄호 `{}` 대신 들여쓰기로 코드 블록을 구분합니다. 왜? 어차피 사람은 들여쓰기로 코드를 읽으니까, 그걸 문법으로 강제한 것입니다.
- **변수 타입을 선언하지 않는다**: `int x = 10` 대신 `x = 10`이면 됩니다. 왜? 파이썬이 알아서 판단하니까.

- **줄 끝에 세미콜론이 필요 없다**: 줄바꿈이 문장의 끝입니다. 왜? 사람이 말할 때 문장 끝에 "세미콜론"이라고 말하지 않으니까.
  (예외: 한 줄에 여러 문장을 쓸 때만 `x = 1; y = 2` 처럼 사용 가능 — 권장되지는 않음)

WHY — 이름이 왜 "파이썬"인가?

뱀이 아닙니다! BBC 코미디 쇼 "Monty Python's Flying Circus"에서 따온 이름입니다. Python 공식 문서에서도 "문서에서 Monty Python의 농담을 인용하는 것은 허락된 것일 뿐만 아니라, 권장되고 있습니다"라고 말합니다[1].
프로그래밍이 재미있어야 한다는 철학이 이름에도 담겨있습니다.

고급

파이썬의 기술적 특성

Python의 공식 문서(Tutorial Chapter 1)에서 언급하는 핵심 특성들:

- **인터프리터 언어**: 컴파일-링크 없이 즉시 실행. REPL(Read-Eval-Print Loop) 지원
- **고수준 자료형 내장**: 유연한 배열(list)과 딕셔너리(dict)가 언어 자체에 내장
- **동적 타이핑(Dynamic Typing)**: 변수 선언 불필요, 런타임에 타입 결정
- **확장 가능(Extensible)**: C로 핵심 연산을 작성하고 파이썬에서 호출 가능
- **대규모 표준 라이브러리**: "Batteries Included" 철학 — 파일I/O, 네트워크, GUI까지 기본 제공
- **모든 것이 객체(Object)**: 숫자, 문자열, 함수까지 전부 객체. 이것이 파이썬 설계의 근본

마지막 포인트가 가장 중요합니다. 앞으로 10주간 우리가 계속 확인할 사실:
파이썬에서는 모든 것이 객체(object)입니다. 숫자 `42` 도 객체, 문자열 `"hello"` 도 객체, 심지어 `print` 함수 자체도 객체입니다.

🚨 **문법 실수 1 — 파이썬과 다른 언어의 문법 혼용:**

```
# C/Java 출신이 자주 하는 실수
x = 10;          # ❌ 세미콜론 — 오류는 아니지만 파이썬 스타일이 아님
x = 10           # ✅ 파이썬 스타일

if (x > 5):      # ❌ 조건 주위 소괄호 — 오류는 아니지만 불필요
    print("큰 수")
if x > 5:        # ✅ 파이썬 스타일

# 중괄호로 블록 표현 시도 (SyntaxError)
# if x > 5 {    # ❌ SyntaxError: invalid syntax
#     print(x)
# }

# 들여쓰기 혼용 (tab + space) — 심각한 오류
# def f():
#     x = 1    (스페이스 4개)
#        y = 2  (탭) ← IndentationError!
```

🚨 **문법 실수 2 — 내장 이름(BUILT-IN NAME)을 변수명으로 덮어쓰기:**

```
# ⚠️ 내장 이름은 덮어쓸 수 있지만, 원래 기능을 잃어 위험!
# (예약어(keyword)와 다릅니다 — 예약어는 아예 SyntaxError)
# list = ["사과", "바나나"]   # ❌ list을 덮어씀! 이후 list() 사용 불가
# print = "hello"             # ❌ print을 덮어씀!
# type = "int"                # ❌ type() 함수를 덮어씀!

# 왜 오류가 안 나도 위험한가?
list = ["사과", "바나나", "포도"]   # 오류는 없지만...
try:
    list(["귤", "망고"])   # TypeError: 'list' object is not callable
except TypeError as e:
    print(f"TypeError: {e}")

# ✅ 올바른 변수명
과일 = ["사과", "바나나", "포도"]
my_list = ["사과", "바나나", "포도"]

# 참고: 진짜 예약어(keyword)는 변수명으로 쓰면 SyntaxError
import keyword
print(keyword.kwlist)   # if, for, class 등 — 이것들은 아예 사용 불가
```

전공자

Python: 언어 명세 vs 구현체 — 그리고 왜 CPython인가

WHY — "파이썬"은 언어인가, 프로그램인가?

중요한 구분이 있습니다: **Python 언어 명세(Language Reference)**와 **CPython 구현체(Implementation)**는 다릅니다.
Python Language Reference는 "파이썬이라는 언어가 어떻게 동작해야 하는가"를 정의합니다.
CPython은 그 명세를 C언어로 구현한 **레퍼런스 구현체(reference implementation)**입니다.
같은 파이썬 코드를 다른 구현체로도 실행할 수 있는 이유가 여기 있습니다.

Part A — CPython 구현체와 GIL

Python 구현체들 — 왜 여러 개가 존재하는가?

구현체	기반 언어	왜 만들었는가?	트레이드오프
CPython	C	레퍼런스. 가장 표준적이고 호환성 최고	GIL로 인한 멀티스레딩 제약
PyPy	RPython	속도: JIT 컴파일러로 CPython보다 빠름 (공식 벤치마크 기하평균 약 4.2×[6], 워크로드에 따라 큰 편차)	일부 C 확장 모듈 호환 불가
Jython	Java	JVM에서 실행 → Java 라이브러리와 통합	Python 3 미지원, 느린 발전
MicroPython	C	마이크로컨트롤러(Arduino, Raspberry Pi Pico)에서 파이썬 실행	표준 라이브러리 서브셋만 지원

왜 CPython이 기본인가?
귀도 반 로섬이 직접 C로 작성했고, CPython은 Python Language Reference를 기준으로 구현됩니다. 새 언어 기능(PEP)도 CPython에 먼저 구현된 뒤 명세에 반영되므로, 사실상 CPython이 표준 역할을 합니다.

GIL — 왜 파이썬은 진짜 병렬 실행이 안 되는가?

WHY — GIL(Global Interpreter Lock)이 존재하는 이유

CPython의 메모리 관리는 **참조 카운팅(Reference Counting)**으로 작동합니다.
객체를 가리키는 변수가 하나 생기면 카운트 +1, 사라지면 -1, 0이 되면 메모리에서 해제합니다.
문제: 여러 스레드가 동시에 같은 객체의 참조 카운트를 수정하면 Race Condition(경쟁 조건)이 발생합니다.
CPython의 해결책: GIL — "한 번에 하나의 스레드만 파이썬 바이트코드를 실행한다"는 전역 잠금.
덕분에 참조 카운팅은 Thread-Safe하지만, CPU 집약 작업의 진짜 병렬화는 불가능합니다.
(Python 3.13+에서 opt-in free-threading 실험 도입 — GIL 없는 CPython 진행 중)

Part B — PEP: Python의 진화 메커니즘

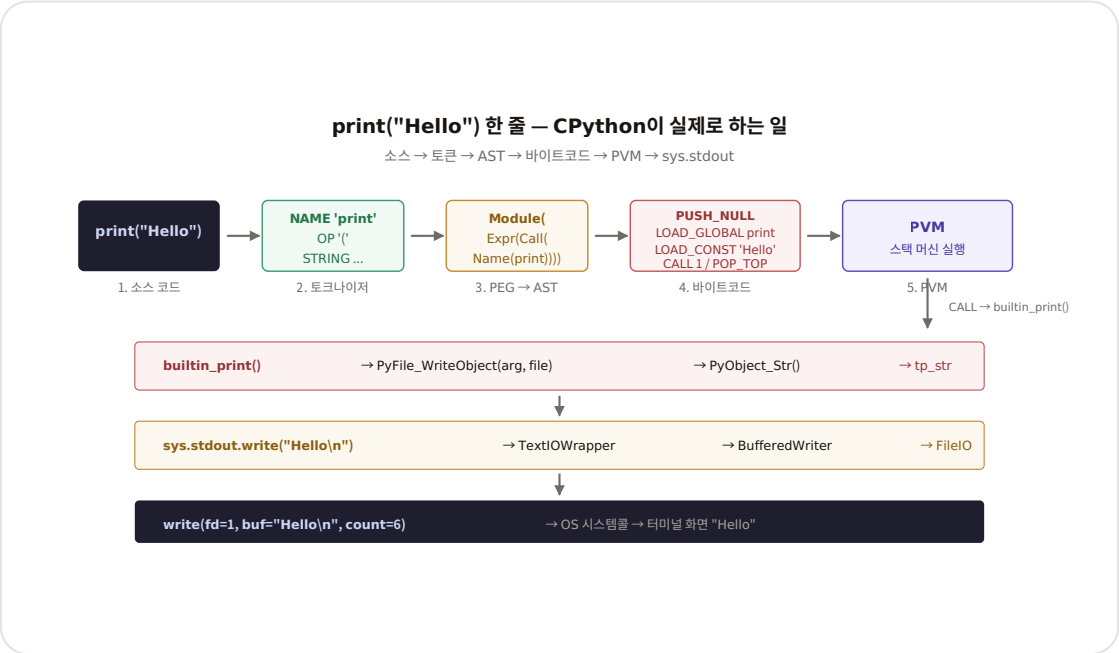
PEP — Python은 어떻게 진화하는가?

PEP(Python Enhancement Proposal)는 파이썬 언어에 변경을 제안하는 공식 문서입니다.
파이썬의 모든 언어 변화에는 PEP 번호가 있습니다:

PEP	내용	왜 이 변화가 필요했는가?
PEP 3105	`print` 를 statement → function으로	언어 일관성 + redirect/suppress 지원
PEP 572	Walrus operator `:=`	while 조건에서 할당 + 검사를 동시에
PEP 634	`match` / `case` 구조적 패턴 매칭	복잡한 데이터 구조 디스패치를 선언적으로
PEP 703	GIL-free CPython (3.13+ opt-in)	멀티코어 병렬화 한계 해소
PEP 8	스타일 가이드	"하나의 명백한 방법" 철학 실현

PEP는 peps.python.org에서 모두 공개됩니다 (소스 저장소: `github.com/python/peps`) [7]. 새 기능이 "왜" 그 형태로 추가됐는지 이해하려면 PEP 원문을 읽는 것이 가장 정확합니다.

print() — 첫 번째 함수를 만나다



초급

화면에 글씨를 보여주는 명령

파이썬에서 화면에 무언가를 보여주고 싶으면 `print()` 를 사용합니다.

첫 번째 파이썬 코드

```
print("안녕하세요!")
```

실행 결과:

```
안녕하세요!
```

`print` 는 "출력해라"라는 뜻이고, 괄호 안에 보여주고 싶은 것을 넣습니다.

✏️ **지금 바로 해봐:**

파이썬 설치 후 아래 코드를 입력하고 Enter를 눌러보세요.

```
print("안녕하세요, 파이썬!")
print(1 + 2)
print("내 이름은", "파이썬")
```

결과로 3줄이 출력되면 성공입니다!

중급

print은 "함수"다 — 함수란 무엇인가?

WHY — 왜 print 뒤에 괄호가 있는가?

`print` 는 단순한 명령어가 아닙니다. **함수(function)**입니다.
함수란 **"특정 작업을 수행하는 기계"**입니다.
괄호 `()` 는 **"이 기계를 작동시켜라"**라는 뜻입니다.

- `print` → 함수 자체 (기계가 그냥 서있는 상태)
- `print()` → 함수를 실행 (기계를 작동시킨 상태)

`print` 의 실제 구조를 봅시다 (Python 공식 문서 그대로)[8]:

함수 구조 (FUNCTION SIGNATURE)

```
print(*objects, sep=' ', end='\n', file=None, flush=False)
```

이게 뭘 의미하는지 하나씩 봅시다:

매개변수	의미	기본값
`*objects`	출력할 것들 (여러 개 가능)	없음
`sep`	여러 개를 출력할 때 사이에 넣을 것	`' '` (공백)
`end`	출력 끝에 붙일 것	`'\n'` (줄바꿈)
`file`	출력할 곳 (파일 객체)	`None` (= `sys.stdout`)
`flush`	출력 버퍼를 즉시 비울지 여부	`False`

※ `file=None`이 기본값인 이유: 공식 문서가 "file 인자가 존재하지 않거나 None이면, sys.stdout이 사용됩니다"라고 명시합니다[8]. `None`으로 두면 인터프리터 시작 시점의 `sys.stdout`을 매번 동적으로 따라갑니다.

print의 숨겨진 기능들

```
# 여러 개 출력 — sep(구분자)이 공백으로 자동 삽입
print("이름:", "파이썬", "나이:", 35)

# sep 바꾸기
print("2025", "03", "30", sep="-")

# end 바꾸기 — 줄바꿈 대신 다른 것
print("로딩", end="...")
print("완료!")
```

실행 결과:

```
이름: 파이썬 나이: 35
2025-03-30
로딩...완료!
```

WHY — 왜 이런 구조인가?

`print`가 매번 `sep=' '`과 `end='\n'`을 기본값으로 갖는 이유:
대부분의 경우 우리는 값들 사이에 공백을 넣고, 줄 끝에서 줄바꿈을 원합니다.
파이썬은 **가장 흔한 사용 방식을 기본값(default)**으로 설정합니다.
이것이 "명시적인 것이 암시적인 것보다 낫다"는 철학의 실현입니다 — 기본값이 뭔지 알면, 안 써도 됩니다. 하지만 바꿀 수도 있습니다.

주의 — 파이썬 2와 파이썬 3의 차이: print

`print()`에 괄호가 있는 이유를 묻는다면, 이 역사적 변화를 알아야 합니다.

```
파이썬 2 (구버전 — 지금은 사용 X)

print "Hello"      # 파이썬 2: print은 "문"(statement)이었다
```

```
파이썬 3에서 위 코드를 실행하면?

>>> print "Hello"
  File "<stdin>", line 1
    print "Hello"
        ^^^^^^^
SyntaxError: Missing parentheses in call to 'print'.
Did you mean print(...)?
```

왜 바뀌었는가?
파이썬 2에서 `print`는 특별한 "문(statement)"이었습니다. 하지만 이것은 파이썬의 핵심 철학인 "모든 것은 객체"에 위배됩니다.
파이썬 3에서는 `print`를 완전한 **함수(function) 객체**로 바꿔 `sep`, `end`, `file` 같은 매개변수를 지원하게 되었습니다.
괄호를 쓰는 것은 단순한 문법 변화가 아니라 **설계 철학의 일관성**을 위한 결정입니다.

🚨 **흔한 실수 1 — 대소문자:**

파이썬은 대소문자를 완전히 다르게 인식합니다.

```
Print("안녕하세요")   # ❌ NameError: name 'Print' is not defined
PRINT("안녕하세요")   # ❌ 똑같이 오류
print("안녕하세요")   # ✅ 소문자 print만 작동
```

🚨 **흔한 실수 2 — 들여쓰기(INDENTATIONERROR):**

파이썬은 빈칸(공백)의 개수가 틀리면 바로 오류가 납니다.

```
나이 = 15
  print(나이)   # ❌ IndentationError: unexpected indent
              # 아무 이유 없이 앞에 공백을 넣으면 안 됨!
print(나이)     # ✅ 정상
```

🚨 **흔한 실수 3 — 따옴표 짝이 안 맞음:**

```
print("안녕')   # ❌ SyntaxError: 시작은 "인데 끝은 '
print("안녕")   # ✅ 양쪽 같은 따옴표
print('안녕')   # ✅ 이것도 정상
```

🚨 **흔한 실수 4 — 문자열과 숫자를 +로 연결:**

```
# ❌ 될 것 같은데 안 되는 코드
print("내 점수: " + 95)
# TypeError: can only concatenate str (not "int") to str

# 왜? + 연산자는 양쪽이 같은 타입이어야 합니다.
# "내 점수: "는 문자열(str), 95는 숫자(int)
# 파이썬은 자동으로 변환해주지 않습니다!

# ✅ 올바른 방법
print("내 점수:", 95)          # 쉼표로 구분 (sep 자동 삽입)
print("내 점수: " + str(95))   # str()로 직접 변환
```

🚨 **흔한 실수 5 — SEP에 인자가 1개면 아무 효과 없음:**

```
# "될 것 같은데" 아무 일도 안 일어나는 코드
print("안녕하세요", sep="---")
# 출력: 안녕하세요
# sep="---"를 넣었는데 왜 "---"가 안 보이지?

# 왜? sep은 여러 인자 "사이"에 삽입됩니다.
# 인자가 1개뿐이면 "사이"가 없으므로 sep은 아무 효과 없음!

# ✅ sep이 작동하려면 인자가 2개 이상
print("짜장면", "짬뽕", "탕수육", sep=" | ")
# 출력: 짜장면 | 짬뽕 | 탕수육
```

🚨 **흔한 실수 6 — SEP에 숫자를 넣으면 TYPEERROR:**

```
# ❌ 될 것 같은데 안 되는 코드
print(1, 2, 3, sep=0)
# TypeError: sep must be None or a string, not int

# 왜? sep의 타입은 반드시 문자열(str)이어야 합니다.
# print()는 내부적으로 file.write(sep)을 호출하는데,
# write()는 문자열만 받기 때문입니다.

# ✅ 올바른 방법
print(1, 2, 3, sep="0")      # 문자열 "0" → 10203
print(1, 2, 3, sep=str(0))   # 같은 결과
```

🚨 **흔한 실수 7 — PRINT() 빈 괄호는 "아무것도 안 하는 것"이 아님:**

```
# "아무것도 안 넣었으니 아무 일도 안 일어날 것 같은데"
print("시작")
print()            # ← 빈 줄이 출력된다!
print("끝")

# 출력:
# 시작
#          ← 여기에 빈 줄!
# 끝

# 왜? print()의 기본값을 보면:
# print(*objects, sep=' ', end='\n')
# objects가 비어있어도 end='\n'은 여전히 실행됩니다.
# 즉 print()는 "아무것도 출력하지 않고 줄바꿈만 하는" 동작!
```

고급

print은 객체다 — 모든 것이 객체

print 자체를 조사해보기

```
# print은 함수 "객체"다
print(type(print))   # <class 'builtin_function_or_method'>

# 함수를 변수에 저장할 수 있다 (일급 객체)
my_print = print
my_print("이것도 출력됩니다!")

# 함수도 id(메모리 주소)를 가진다
print(id(print))
```

실행 결과:

```
<class 'builtin_function_or_method'>
이것도 출력됩니다!
```

```
140234866532640
```

`print`는 파이썬이 시작될 때 자동으로 만들어놓는 **내장 함수(built-in function) 객체**입니다.
`type()`으로 확인하면 "함수 타입"이고, `id()`로 확인하면 메모리 주소를 가진 실체입니다.
변수에 저장할 수도 있습니다 — 파이썬에서 함수는 **일급 객체(first-class object)**이기 때문입니다.

이 개념은 9주차 함수편에서 깊이 다루지만, 지금부터 기억해두세요:
파이썬에서 `print`는 "명령어"가 아니라 "객체"입니다.

전공자

PEP 3105: print이 함수가 된 이유 + sys.stdout 내부 구조

WHY — PEP 3105: "Make print a function"

파이썬 2에서 `print`는 statement(문)였습니다. Statement는 언어의 특별한 문법 요소이고, Function은 일반 객체입니다.
왜 이것이 문제였는가?

- **출력 목적지 변경 불가**: `print >> sys.stderr, "msg"` 같은 hack이 필요했음
- **다른 함수에 전달 불가**: `map(print, items)`처럼 함수로 넘길 수 없었음
- **일관성 파괴**: `print`만 특별 취급 → "모든 것은 객체" 철학 위반
- **테스트/mocking 어려움**: statement는 override할 수 없음

PEP 3105 (Georg Brandl, 2006)[9]는 이 모든 문제를 함수화로 해결했습니다.

print()는 실제로 무엇을 호출하는가?

print()의 실제 동작 — sys.stdout 레이어

```
import sys

# print("안녕하세요")는 내부적으로 이것과 동등하다:
sys.stdout.write("안녕하세요")
sys.stdout.write("\n")   # end='\n' 기본값

# 여러 인자의 경우 print("짜장면", "짬뽕", sep="-") 는:
sys.stdout.write("짜장면")
```

```
sys.stdout.write("-")    # sep
sys.stdout.write("짬뽕")
sys.stdout.write("\n")   # end
```

sys.stdout는 교체 가능하다 — 왜 이것이 설계에 중요한가?

출력을 파일로 redirection — print의 file= 매개변수와 동일한 원리

```
import sys

# 방법 1: file= 매개변수
print("에러 메시지", file=sys.stderr)

# 방법 2: sys.stdout 자체를 교체 (monkey-patching)
original_stdout = sys.stdout
with open("output.txt", "w") as f:
    sys.stdout = f
    print("이것은 파일로 갑니다")  # 화면이 아닌 output.txt에 기록
sys.stdout = original_stdout
print("이건 다시 화면으로")
```

OUTPUT.TXT 파일 내용:

```
이것은 파일로 갑니다
```

왜 이것이 가능한가?
파이썬 2에서 `print`는 항상 `sys.stdout`에 하드코딩되어 있었습니다. 파이썬 3에서 `print(file=...)`를 추가하면서, 출력 목적지가 완전히 제어 가능해졌습니다.
이것이 **단위 테스트에서 print 출력을 캡처**하거나, **로깅 시스템과 통합**하거나, **GUI 텍스트 위젯으로 리디렉션**할 수 있는 이유입니다.

Part A — CPython 내부: print()의 구현 경로 (builtin_print → tp_str)

`print()`가 실행될 때 CPython 내부에서 어떤 일이 일어나는지 역추적합니다.

print("hello")의 바이트코드 — dis 모듈로 확인

```
import dis

# print("hello")의 바이트코드
def call_print():
    print("hello")

dis.dis(call_print)
```

실행 결과 (PYTHON 3.12+ 기준, 이해를 위해 개별 명령어로 분리 표현):

```
  RESUME              0
  PUSH_NULL                           ← NULL을 스택에 푸시 (메서드가 아닌 함수 호
출 표시)
  LOAD_GLOBAL         print           ← 내장 함수 print 로드
  LOAD_CONST          'hello'         ← 인자 로드
  CALL                1               ← 함수 호출
  POP_TOP                             ← 반환값 None 버리기
  RETURN_CONST        None
```

print()의 내부 경로 — builtin_print → PyObject_Str → tp_str

```
import dis

# print()가 내장 함수임을 확인
print(type(print))           # <class 'builtin_function_or_method'>
print(print.__module__)      # builtins
print(print.__qualname__)    # print

# CPython 구현 경로:
# 1. CALL 바이트코드 → ceval.c에서 함수 호출 처리
# 2. builtin_print() in Python/bltinmodule.c
# 3. 각 인자에 대해 PyObject_Str() 호출 → tp_str 메서드
# 4. sep으로 구분하여 file.write() 호출
# 5. end 출력 후 flush (필요시)

# str() 변환 과정 추적
```

```
class MyObj:
    def __str__(self):
        print("  __str__ 호출됨!")
        return "MyObj"
    def __repr__(self):
        print("  __repr__ 호출됨!")
        return "MyObj()"

obj = MyObj()
print(obj)   # print()는 __str__을 먼저 호출
# → __str__ 호출됨!
# → MyObj

# print()의 실제 시그니처
help(print)  # print(*args, sep=' ', end='\n', file=None, flush=False)
             # (file=None은 sys.stdout을 의미 — 공식 문서 기준)
```

CPYTHON 소스 (PYTHON/BLTINMODULE.C):

`builtin_print()` 는 각 인자에 대해 `PyFile_WriteObject(arg, file, Py_PRINT_RAW)` 를 호출합니다. `Py_PRINT_RAW` 플래그는 `repr()` 대신 `str()` 을 사용하라는 의미입니다. 이것이 `print("hello")` 가 `'hello'` (따옴표 포함)가 아닌 `hello` 를 출력하는 이유입니다.

Part B — sys.stdout 버퍼링과 flush

`print()` 는 실제로 `sys.stdout.write()` 를 호출합니다. stdout은 기본적으로 **라인 버퍼(line-buffered)**로 동작합니다. 터미널에서는 줄바꿈(`\n`) 시 자동 flush, 파이프/리다이렉션에서는 블록 버퍼.

sys.stdout 버퍼링과 flush 예제

```
import sys

# print()의 실제 동작
print("안녕하세요")
# ↕ 아래와 완전히 동일
sys.stdout.write("안녕하세요\n")

# flush=True: 버퍼를 즉시 비워 출력 (진행률 표시 등에 필수)
import time
```

```
for i in range(5):
    print(f"\r진행: {i+1}/5", end="", flush=True)
    time.sleep(0.5)
print()  # 줄바꿈

# file= 매개변수: 출력 대상 변경
print("에러 발생!", file=sys.stderr)  # stderr로 출력

# sep=, end= 매개변수의 의미
print("사과", "바나나", "포도", sep="-", end="!\n")  # 사과-바나나-포도!
```

실행 결과:

```
사과-바나나-포도!
```

CPYTHON 내부:

`print()` 는 `builtin_print()` (Python/bltinmodule.c)에서 구현됩니다. 내부적으로 `PyFile_WriteObject()` → `PyObject_Str()` → `tp_str` 메서드를 호출합니다. `sep` 의 기본값은 `NULL` (C 레벨)이며 Python에서 `" "` 로 처리됩니다.

CPython 소스에서 print의 실제 구현

CPython 소스의 `Python/bltinmodule.c` 에서 `builtin_print()` 함수가 구현됩니다. 핵심 로직을 의사코드로 보면:

builtin_print() 의사코드 (실제 CPython 로직 기반)

```
# CPython Python/bltinmodule.c 의 builtin_print() 핵심 로직
# (실제 코드는 C이지만, 파이썬으로 표현하면)

def print(*objects, sep=' ', end='\n', file=None, flush=False):
    if file is None:
        file = sys.stdout          # 기본 출력 대상

    for i, obj in enumerate(objects):
        if i > 0:
            file.write(sep)        # 구분자 삽입
        file.write(str(obj))       # str() 변환 후 write
```

```
    file.write(end)                # 끝 문자

    if flush:
        file.flush()               # 버퍼 강제 비우기
```

`str(obj)` 를 호출한다는 것을 주목하세요. 이것은 **어떤 객체든** `__str__()` 메서드가 있으면 print할 수 있다는 의미입니다. (9주차 함수 + 이후 클래스 주차에서 `__str__` 의 역할을 다룹니다)

__builtins__ — print은 어디서 오는가?

```
print의 출처 추적

import builtins

# print은 builtins 모듈의 속성이다
print(print is builtins.print)   # True

# 파이썬이 시작되면, builtins의 모든 이름이
# 모든 모듈의 '__builtins__'에 자동으로 삽입된다
print(type(__builtins__))        # <class 'module'> 또는 dict
print('print' in dir(__builtins__))  # True

# 이것이 import 없이 print을 쓸 수 있는 이유다
# LEGB에서 B(Built-in)가 바로 이 builtins 모듈을 의미한다
```

파이썬의 이름 조회 규칙 **LEGB**(Local → Enclosing → Global → Built-in)에서 **B(Built-in)**가 바로 `builtins` 모듈입니다. `print` 를 import 없이 쓸 수 있는 이유는, 파이썬이 시작할 때 `builtins` 를 자동으로 모든 네임스페이스에 노출하기 때문입니다.

AI는 코딩을 어떻게 하는가?

초급

AI = 엄청나게 많이 읽은 학생

AI(인공지능)는 **수십억 개의 코드**를 읽고 패턴을 학습한 프로그램입니다.

여러분이 "사과가 3개 있고 2개를 더 사면 몇 개?"라는 문제를 풀 수 있는 이유는,
비슷한 문제를 많이 풀어봤기 때문입니다. AI도 같습니다.

바이브 코딩(Vibe Coding)이란?
AI에게 "이런 거 만들어줘"라고 말하면, AI가 코드를 대신 짜주는 것입니다.
마치 "이런 느낌(vibe)으로 해줘"라고 말하는 것과 같아서 바이브 코딩이라고 합니다.

중급

AI가 코드를 생성하는 원리

AI(대규모 언어 모델, LLM)가 코드를 작성하는 과정:

1. **학습**: GitHub의 수십억 줄 코드 + Stack Overflow 질답 + 공식 문서를 읽음
2. **패턴 인식**: "이런 요청이 오면 이런 코드가 나온다"는 패턴을 학습
3. **생성**: 사용자의 요청(프롬프트)을 받으면, 학습한 패턴에 따라 가장 **그럴듯한** 다음 토큰을 예측

사람이 코딩할 때

1. 문제를 이해한다
2. 해결 방법을 설계한다
3. 코드를 작성한다
4. 테스트하고 수정한다

AI가 코딩할 때

1. 프롬프트(요청)를 받는다
2. 비슷한 패턴을 찾는다
3. 확률적으로 코드를 생성한다
4. **스스로 테스트하지 않는다**

주의 — AI 할루시네이션: 실제 사례

AI는 "이해"하지 않습니다. **패턴 매칭**을 합니다.
그래서 AI가 만든 코드가 "그럴듯해 보이지만 틀린" 경우가 자주 있습니다.
이것을 **할루시네이션(hallucination)**이라고 합니다.

실제 AI 할루시네이션 예시:

사용자가 AI에게 질문합니다: "사용자에게 나이를 입력받아서 10을 더해서 출력해줘"

AI가 생성한 코드:

```
AI가 만든 코드 (겉으로는 맞아 보임)
```

```
age = input("나이를 입력하세요: ")
result = age + 10          # AI는 이게 맞다고 출력했다
print("10년 후:", result)
```

실제 실행하면? → `TypeError: can only concatenate str (not "int") to str`

왜 틀렸는가? `input()` 은 항상 **문자열(str)**을 반환합니다.
사용자가 `20` 을 입력해도 파이썬은 그것을 문자열 `"20"` 으로 받습니다.
그런데 AI는 학습 데이터에서 "입력받은 값에 숫자를 더하는" 패턴을 봤기 때문에,
`int(input(...))` 로 변환해야 한다는 사실을 종종 빠뜨립니다.

올바른 코드: `age = int(input("나이를 입력하세요: "))`

이 차이를 아는 사람만이 AI가 만든 코드를 **검증**할 수 있습니다.
이것이 AI 시대에도 코딩을 배워야 하는 가장 현실적인 이유입니다.

고급

바이브 코딩 시대의 개발자 역할

AI가 코드를 짜주는 시대에 개발자의 역할은 변하고 있습니다:

과거의 개발자	AI 시대의 개발자
코드를 직접 타이핑	AI에게 정확한 지시(프롬프트) 작성
문법을 외움	코드의 **동작 원리**를 이해
모든 코드를 직접 작성	AI가 생성한 코드를 **읽고, 검증하고, 수정**
구현 능력이 핵심	**설계 능력과 판단력**이 핵심

이 강의가 "파이썬 문법"이 아닌 **"파이썬이 왜 이렇게 동작하는가"**에 집중하는 이유:
AI에게 "리스트를 정렬해줘"라고 시킬 수는 있지만,
AI가 반환한 코드가 **왜** 그렇게 생겼는지 이해해야 그것이 맞는지 판단할 수 있기 때문입니다.

전공자

LLM이 코드를 "생성"하는 메커니즘 — 왜 이해가 아닌 확률인가

WHY — LLM과 Python 인터프리터의 근본적 차이

Python 인터프리터는 코드를 **문법적으로 파싱**하고 **의미론적으로 실행**합니다 — 규칙 기반, 결정론적.
LLM은 코드를 **확률적으로 생성**합니다 — 통계 기반, 비결정론적.
이 근본적 차이가 AI 코드 검증 능력이 왜 중요한지를 설명합니다.

Part A — LLM의 토크나이저와 생성 메커니즘

LLM 토크나이저와 Python 토크나이저의 차이

Python 인터프리터의 토크나이저는 **결정론적 규칙**으로 작동합니다: `print`는 항상 NAME 토큰, `"Hello"`는 항상 STRING 토큰입니다.
반면 LLM(GPT-4, Claude 등)은 **BPE(Byte Pair Encoding)** 토크나이저를 씁니다:

Python 코드가 LLM 토크나이저에서 분해되는 방식 (tiktoken 기반)

```
# Python 인터프리터 토크나이저: 언어 문법 기반
# print("Hello")  →  [NAME:'print'] [OP:'('] [STRING:'"Hello"'] [OP:')']

# GPT 계열 LLM BPE 토크나이저 (tiktoken 기준):
# print("Hello")  →  ["print", '("', "Hello", '")']
# — 따옴표와 괄호가 붙어서 하나의 토큰이 될 수 있음
# — 들여쓰기(indentation)는 공백 개수에 따라 다른 토큰
# — "    " (4칸) vs "   " (3칸)은 다른 토큰 → LLM이 들여쓰기 실수하는 원인

# 확인해보기 (tiktoken 설치 필요: pip install tiktoken)
import tiktoken
enc = tiktoken.encoding_for_model("gpt-4o")
tokens = enc.encode('print("Hello")')
print([enc.decode([t]) for t in tokens])
# 결과는 모델·버전마다 다름
```

Transformer의 Next Token Prediction — "이해"가 아닌 이유

LLM이 코드를 생성하는 수식:

LLM의 코드 생성 원리 (수식으로)

```
# LLM은 이 확률을 최대화하는 토큰을 순서대로 선택합니다:
# P(t_n | t_1, t_2, ..., t_{n-1})
# = "이전 토큰들이 주어졌을 때, 다음 토큰이 t_n일 확률"
#
# 즉 LLM은 다음 질문에 답합니다:
# "지금까지 본 코드 문맥에서, 다음에 올 가능성이 가장 높은 텍스트 조각은?"
#
# Python 인터프리터가 묻는 질문:
# "이 코드가 문법적으로 valid한가? 의미론적으로 무엇을 해야 하는가?"
#
# → 두 질문은 완전히 다릅니다
# → LLM이 "그럴듯해 보이지만 틀린" 코드를 내놓는 이유
```

Part B — 형식 언어 이론과 LLM의 한계

Python 문법은 형식 언어 — LLM은 이를 보장하지 않는다

WHY — Python 문법은 형식 언어, LLM 출력은 확률 분포

Python의 공식 문법은 **PEG(Parsing Expression Grammar, 파싱 표현 문법)**로 정의됩니다 — Python 3.9부터 PEP 617에 의해 LL(1) 파서에서 PEG로 전환되었습니다 [10].
PEG 파서는 모든 유효한 프로그램을 정확히 인식하고, 모든 무효한 프로그램을 정확히 거부합니다 — **SyntaxError는 이 거부의 결과**입니다.

※ PEG는 흔히 알려진 Context-Free Grammar(CFG)와 유사하지만 표현력이 더 넓습니다 (CFG로 표현 어려운 일부 문맥 의존성을 ordered choice로 자연스럽게 표현). Chomsky 계층에 깔끔히 매핑되지는 않습니다.

LLM은 이 형식 문법을 직접 구현하지 않습니다. 학습 데이터에서 "올바른 파이썬 코드"를 많이 봤기 때문에 통계적으로 문법을 근사할 뿐입니다.
그래서:

- 괄호를 닫지 않은 코드를 생성할 수 있습니다 (`print("hello"`)
- 존재하지 않는 함수를 확신 있게 쓸 수 있습니다 (`df.get_column_names()`)
- 들여쓰기를 3칸으로 맞추는 실수를 할 수 있습니다
- Python 2 문법을 Python 3 코드에 섞을 수 있습니다

Grammar-Constrained Generation — 최신 해결책

이 문제를 해결하기 위한 연구 방향:

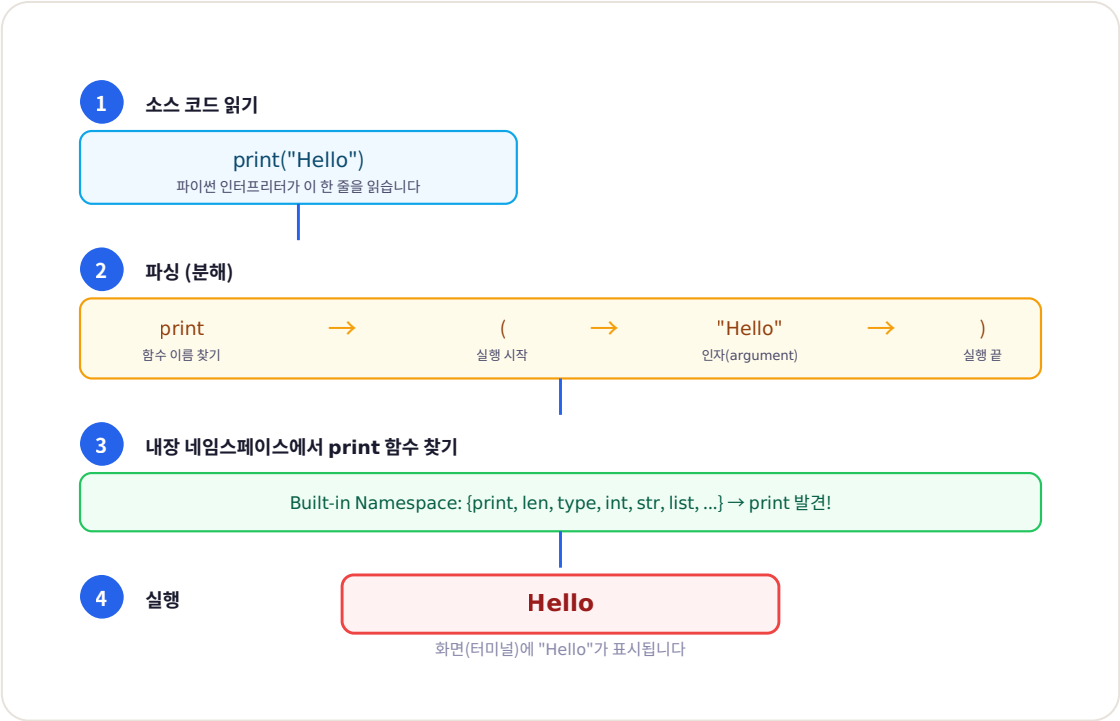
방법	원리	한계
문법 제약 생성 (Grammar-Constrained)	생성 중에 Python 공식 문법(PEG)을 실시간 파싱하여 문법적으로 불가능한 토큰의 확률을 0으로 마스킹	문법은 보장하지만 의미론(semantics)은 여전히 보장 못함
실행 피드백 (Execution Feedback)	생성한 코드를 실제 실행해보고, 에러가 나면 에러 메시지를 context에 넣어 재생성	실행 환경 필요, 무한 루프·사이드이펙트 위험
타입 검사 통합	mypy/pyright 결과를 피드백으로 제공	타입 힌트가 없는 코드에 한계

결론: 어떤 방법을 쓰더라도 현재의 LLM은 "코드를 이해하고 검증"하는 것이 아니라 "코드처럼 생긴 텍스트를 확률적으로 생성"합니다.
Python 실행 모델(인터프리터, 타입 시스템, 스코프 규칙)을 이해하는 개발자만이 LLM 출력의 옳고 그름을 판단할 수 있습니다. 이것이 이 강의의 핵심 동기입니다.

컴퓨터는 print("Hello")를 어떻게 처리하는가?

컴퓨터의 관점에서 보기

우리가 `print("Hello")` 라고 쓰면, 컴퓨터 내부에서는 이런 일이 벌어집니다:



단 한 줄의 코드 `print("Hello")` 뒤에는 이런 과정이 숨어있습니다.
앞으로 우리는 매주, 코드 한 줄 뒤에 숨은 **"왜?"**를 파헤칠 것입니다.

전공자 **완전한 실행 스택: Python 코드 → 커널 → 화면**

Part A — CPython에서 커널까지: 전체 실행 스택

앞의 4단계 다이어그램은 파이썬 수준의 이야기였습니다. 실제로는 운영체제 커널까지 내려갑니다.

전체 실행 스택 — print("Hello")가 화면에 나타나기까지

```
레이어별 처리 과정

Layer 0: 소스 코드
  print("Hello")

Layer 1: CPython 컴파일러 (토크나이저 → 파서 → AST → 바이트코드)
  PUSH_NULL
  LOAD_GLOBAL     print        ← builtins에서 print 객체 로드
  LOAD_CONST      'Hello'      ← 상수 풀에서 문자열 로드
  CALL            1            ← print 호출
```

```
  POP_TOP                          ← None 버림

Layer 2: CPython PVM → builtin_print() 실행 (C 함수)
  str("Hello") → "Hello"
  sys.stdout.write("Hello")
  sys.stdout.write("\n")

Layer 3: sys.stdout → TextIOWrapper (파이썬 I/O 레이어)
  → BufferedWriter (버퍼: io.DEFAULT_BUFFER_SIZE만큼 모은 후 한 번에 씀
                    — Python 3.13 이하: 8KB / 3.14+: 128KB, gh-117151)
  → FileIO (파이썬 파일 객체)

Layer 4: OS 시스템콜
  write(fd=1, buf="Hello\n", count=6)
  ↑ fd=1 은 stdout (File Descriptor 1)
  ↑ POSIX: 0=stdin, 1=stdout, 2=stderr

Layer 5: OS 커널
  → VFS (Virtual File System)
  → 터미널 드라이버 (tty/pty)

Layer 6: 터미널 에뮬레이터
  → 화면에 "Hello" 렌더링
```

Part B — I/O 버퍼링과 File Descriptor

왜 버퍼(Buffer)가 존재하는가?

WHY — I/O 버퍼링: 왜 즉시 화면에 안 나타날 수 있는가?

시스템콜(`write()`)은 비용이 큽니다. 커널 모드로 전환해야 하기 때문입니다.
`print("A")`를 1000번 호출할 때마다 시스템콜을 1000번 하면 매우 느립니다.
그래서 파이썬의 `sys.stdout`은 기본적으로 **버퍼링**을 합니다:
데이터를 메모리에 쌓다가 버퍼가 가득 차거나 `flush()`가 호출될 때 한 번에 씁니다.

이것이 `print(flush=True)`가 존재하는 이유입니다:
진행 표시줄처럼 즉시 화면에 나타나야 할 때는 버퍼를 즉시 비워야 합니다.

버퍼링 차이 확인

```
import time

# flush=False (기본): 줄바꿈이 없으면 버퍼에 쌓임 → 지연 후 한꺼번에 출력
for i in range(5):
    print(f"처리 중... {i}", end="\r")  # \r = 커서를 줄 처음으로
    time.sleep(0.5)                     # 버퍼링 때문에 즉시 안 나타날 수 있음

# flush=True: 버퍼를 즉시 비워서 실시간 출력 보장
for i in range(5):
    print(f"처리 중... {i}", end="\r", flush=True)
    time.sleep(0.5)
```

File Descriptor — 왜 0, 1, 2인가?

UNIX/Linux/macOS에서 프로세스가 시작될 때 OS는 자동으로 3개의 파일 디스크립터를 엽니다:

FD	이름	파이썬	기본 연결
0	stdin (표준 입력)	`sys.stdin`	키보드
1	stdout (표준 출력)	`sys.stdout`	터미널 화면
2	stderr (표준 에러)	`sys.stderr`	터미널 화면 (별도)

`print("Hello")`가 화면에 나타나는 것은: `sys.stdout` → FD 1 → 터미널 에뮬레이터 → 화면 렌더링의 연결이기 때문입니다.
이것이 `print(file=sys.stderr)`로 에러를 별도 채널로 보낼 수 있는 이유입니다.
셸에서 `python script.py 2>/dev/null`로 에러만 숨길 수 있는 것도 같은 원리입니다.

실습: 파이썬 첫 실행

방법 1: 설치 없이 바로 시작 (Google Colab)

인터넷만 되면 바로 파이썬을 실행할 수 있습니다.
colab.research.google.com에 접속 → "새 노트" 클릭 → 코드 입력 → 실행(Shift+Enter)

방법 2: 내 컴퓨터에 설치

1. **python.org**에서 최신 버전 다운로드

2. 설치할 때 **"Add Python to PATH"** 반드시 체크
3. 터미널(명령 프롬프트)에서 `python` 입력
4. `>>>` 프롬프트가 뜨면 성공!

```
터미널에서 확인

python --version
```

```
실행 결과 (예시):

    Python 3.14.0
```

REPL — 파이썬과 직접 대화하기

Python 공식 문서 (Tutorial 2장 — Using the Python Interpreter)

"When commands are read from a tty, the interpreter is said to be in interactive mode. In this mode it prompts for the next command with the primary prompt, usually three greater-than signs (`>>>`)."
— Python 3.14 공식 문서, Chapter 2.1.2[11]

초급 **파이썬은 대화할 수 있다**

파이썬에는 두 가지 실행 방법이 있습니다:

방법	어떻게	비유
대화형 (REPL)	터미널에서 `python` 입력 후 한 줄씩 입력	계산기 — 누를 때마다 바로 결과
스크립트	`.py` 파일 저장 후 `python 파일명.py` 실행	레시피 — 전체를 처음부터 끝까지 실행

터미널에서 `python`을 입력하면 나타나는 `>>>`가 바로 **REPL 프롬프트**입니다. 여기에 코드를 입력하면 즉시 결과가 나옵니다.

REPL 대화 예시 (>>> 는 직접 입력, 나머지는 파이썬의 답변)

```
>>> print("안녕하세요!")
안녕하세요!
>>> 1 + 1
2
>>> "파이썬" + "은 왜?"
'파이썬은 왜?'
```

REPL은 Read(읽기) → Eval(계산) → Print(출력) → Loop(반복)의 약자입니다.
여러분이 코드를 입력하면(**Read**), 파이썬이 계산하고(**Eval**), 결과를 보여주고(**Print**), 다음 입력을 기다립니다(**Loop**).

중급

REPL의 구체적 동작 + 주석(#) + help()/dir()

주석(Comment) — 파이썬이 무시하는 줄

`#` 기호로 시작하는 줄은 파이썬이 무시합니다. 사람을 위한 설명입니다.

주석 예시 (Python 공식 문서 introduction.txt 기반)

```
# 이건 주석입니다 — 파이썬이 이 줄을 실행하지 않습니다
print("오늘 날씨 좋다")   # 줄 끝에도 주석을 쓸 수 있습니다
# print("이건 실행 안 됩니다")
```

실행 결과:

```
오늘 날씨 좋다
```

REPL의 특별한 기능 — 마지막 값 자동 출력

REPL에서는 `print()` 없이도 표현식의 값이 출력됩니다. 하지만 파일에서는 반드시 `print()`를 써야 합니다.

REPL vs 스크립트 차이

```
# REPL에서 (print 없이도 출력됨)
>>> 2 + 3
```

```
5
>>> "hello"
'hello'

# 파일(.py)에서 실행 시 — 아무것도 출력 안 됨!
2 + 3       # 이건 화면에 아무것도 안 나옵니다
print(2 + 3)  # 이래야 화면에 나옵니다
```

🚨 **함정 — REPL에서 따옴표가 보이는 이유: REPR() VS STR()**

```
# REPL에서 실행해보세요:
>>> "안녕하세요"
'안녕하세요'          # ← 따옴표가 있다!

>>> print("안녕하세요")
안녕하세요            # ← 따옴표가 없다!

# 왜 다른가?
# REPL의 자동 출력은 repr()을 호출 → 따옴표 포함 ("이건 문자열이다" 표시)
# print()는 str()을 호출 → 따옴표 없이 내용만 표시

# 이것은 "표현(representation)"과 "표시(display)"의 차이입니다.
# repr(): 개발자를 위한 정확한 표현 (타입 정보 포함)
# str():  사용자를 위한 읽기 좋은 표시
```

파이썬은 자기 자신을 설명한다 — help()와 dir()

WHY — 왜 help()와 dir()가 존재하는가?

파이썬의 철학: "Batteries Included" — 필요한 것이 이미 다 있다.
`help()`는 내장 문서를 읽어주고, `dir()`는 객체의 모든 기능을 나열합니다.
인터넷 없이도 파이썬 안에서 파이썬을 배울 수 있도록 설계된 것입니다.

REPL에서 파이썬에게 직접 질문하기

```
>>> help(print)
Help on built-in function print in module builtins:

print(*objects, sep=' ', end='\n', file=None, flush=False)
```

```
    Prints the values to a stream, or to sys.stdout by default.
    ... (file=None은 sys.stdout을 의미; 표시 형식은 Python 버전마다 다를 수 있습니다)

>>> dir(print)
['__call__', '__class__', '__doc__', '__module__', '__name__', ...]

>>> print.__doc__[:80]   # print의 설명서 첫 80글자
"print(*objects, sep=' ', end='\\n', file=None, flush=False)\\nPrints the va"
```

`help(print)` 는 `print` 함수의 공식 문서를 REPL 안에서 바로 보여줍니다.
이것은 파이썬이 **자기 서술적(self-documenting)**으로 설계되어 있기 때문입니다.
모르는 함수가 생기면 `help(함수이름)` 을 먼저 쳐보는 습관을 들이세요.

고급

파이썬의 철학 + 바이트코드 직접 보기

import this — 파이썬의 선(禪)

파이썬에는 숨겨진 이스터에그가 있습니다. REPL에서 `import this` 를 입력해보세요:

파이썬의 설계 철학 전체

```
>>> import this
```

실행 결과:

```
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
```

```
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

이 19가지 원칙이 파이썬의 모든 설계 결정의 배경입니다.
`print()`에 괄호가 있는 것도, 들여쓰기가 문법인 것도, 모두 이 철학에서 나왔습니다.

dis 모듈 — 파이썬이 실제로 실행하는 것을 보다

WHY — 왜 바이트코드를 봐야 하는가?

우리가 쓴 파이썬 코드는 직접 실행되지 않습니다.
파이썬 인터프리터는 우리 코드를 먼저 **바이트코드(bytecode)**로 변환한 뒤,
PVM(Python Virtual Machine)이 그것을 실행합니다.
`dis` (disassembler) 모듈은 이 중간 단계를 눈으로 볼 수 있게 해줍니다.

print("Hello")의 바이트코드 분해

```
import dis

def hello():
    print("Hello")

dis.dis(hello)
```

실행 결과 (PYTHON 3.12+):

```
  2           RESUME                   0

  3           PUSH_NULL
```

```
              LOAD_GLOBAL              1 (print)
              LOAD_CONST               1 ('Hello')
              CALL                     1
              POP_TOP
              RETURN_CONST             0 (None)
```

각 줄이 PVM이 실행하는 **하나의 명령**입니다:

- `LOAD_GLOBAL print` → "내장 네임스페이스에서 `print` 라는 이름을 찾아라"
- `LOAD_CONST 'Hello'` → "상수 문자열 `'Hello'` 를 스택에 올려라"
- `CALL 1` → "스택에 있는 함수를 인자 1개로 호출하라"
- `POP_TOP` → "`print` 가 반환한 `None` 을 버려라"

단 한 줄 `print("Hello")` 가 이렇게 여러 단계로 분해됩니다.
이것이 "파이썬 인터프리터가 코드를 어떻게 처리하는가"의 실제 모습입니다.

전공자 **Python 컴파일 파이프라인 전체 — 소스에서 실행까지**

WHY — 파이썬은 "인터프리터"이지만 컴파일도 한다?

"파이썬은 인터프리터 언어"라는 말은 정확하지 않습니다.
정확히는: **파이썬은 소스 코드를 바이트코드로 컴파일한 뒤, PVM이 그 바이트코드를 해석(interpret)합니다.**
컴파일러 언어(C, C++)와의 차이는 컴파일 여부가 아니라, 컴파일 대상이 기계어인가 바이트코드인가입니다.
그리고 바이트코드는 `.pyc` 파일로 캐싱됩니다 — 다음 실행 때 재컴파일을 생략하기 위해.

Part A — CPython 컴파일 파이프라인과 .pyc 캐싱

4단계 컴파일 파이프라인 (Python 3.9+, PEG 파서 기본)

```
파이썬의 전체 컴파일 파이프라인

소스 코드: print("Hello")
    ↓
[1단계] Tokenizer (토크나이저)
```

```
  → NAME 'print'
  → OP '('
  → STRING '"Hello"'
  → OP ')'
  → NEWLINE
  → ENDMARKER

    ↓
[2단계] PEG Parser → AST (Abstract Syntax Tree)
  → PEG 파서(Grammar/python.gram 기반)가 토큰에서 직접 AST 생성
  → Python 3.9(PEP 617)부터 PEG 파서가 기본으로 도입되어 토큰에서 직접 AST 생성
    (Python 3.10에서 구 LL(1) 파서가 완전히 제거됨)
  → ast 모듈로 직접 볼 수 있음 ↓

[3단계] Compiler → Bytecode
  → AST를 CPython 바이트코드 명령어로 변환
  → dis 모듈로 직접 볼 수 있음 ↓

[4단계] PVM (Python Virtual Machine)
  → 바이트코드를 스택 기반으로 실행
  → 결과: 화면에 "Hello" 출력
```

Part B — ast, compile, tokenize로 내부 들여다보기

ast 모듈 — 파이썬 코드의 추상 구문 트리 직접 보기

print("Hello")의 AST 구조

```
import ast

source = 'print("Hello")'
tree = ast.parse(source)
print(ast.dump(tree, indent=2))
```

실행 결과:

```
Module(
  body=[
```

```
    Expr(
      value=Call(
        func=Name(id='print', ctx=Load()),
        args=[
          Constant(value='Hello')
       ],
        keywords=[]))],
  type_ignores=[])
```

AST가 말하는 것: "이것은 `Expr`(표현식 문)이고, 내용은 `Call`(함수 호출)이다. 호출 대상(`func`)은 `Name`(`'print'`)이고, 인자(`args`)는 `Constant`(`'Hello'`)다."
이것이 파이썬이 `print("Hello")`를 "이해"하는 방식입니다.

compile() 내장 함수 — AST → 바이트코드 코드 객체

compile()로 코드 객체 생성 + 내부 확인

```
import dis

source = 'print("Hello")'

# compile(): 소스 → 코드 객체(code object)
code = compile(source, "<string>", "exec")

print(type(code))          # <class 'code'>
print(code.co_consts)      # ('Hello', None) — 상수 풀 (인덱스 0='Hello',
1=None)
print(code.co_names)       # ('print',) — 참조된 이름들

# 이 코드 객체를 dis로 분해:
dis.dis(code)
```

실행 결과:

```
<class 'code'>
('Hello', None)
('print',)
  1           PUSH_NULL
```

```
              LOAD_NAME                0 (print)
              LOAD_CONST               0 ('Hello')
              CALL                     1
              POP_TOP
              RETURN_CONST             1 (None)
```

.pyc 파일 — 왜 __pycache__ 폴더가 생기는가?

WHY — .pyc와 __pycache__의 존재 이유

파이썬이 `hello.py`를 처음 실행하면:

1. 소스 → 바이트코드로 컴파일
2. 컴파일된 바이트코드를 `__pycache__/hello.cpython-312.pyc`에 저장
3. 다음 실행 때 소스가 변경되지 않았으면 `.pyc`를 그대로 사용 (1~3단계 생략)

`cpython-312`는 "CPython 3.12로 컴파일된"을 의미합니다. 다른 파이썬 버전이나 다른 구현체(PyPy 등)와 충돌하지 않도록 버전 정보를 파일명에 포함합니다.
이것이 파이썬이 "인터프리터"임에도 연속 실행이 빠른 이유 중 하나입니다.

tokenize 모듈 — 파이썬의 토크나이저 직접 보기

```
import tokenize, io

source = 'print("Hello")'
tokens = tokenize.generate_tokens(io.StringIO(source).readline)
for tok in tokens:
    print(tok)
```

실행 결과:

```
TokenInfo(type=NAME,      string='print',   ...)
TokenInfo(type=OP,        string='(',       ...)
TokenInfo(type=STRING,    string='"Hello"', ...)
TokenInfo(type=OP,        string=')',       ...)
TokenInfo(type=NEWLINE,   string='',        ...)
TokenInfo(type=ENDMARKER, string='',        ...)
```

`print("Hello")` 가 파이썬 눈에는 이 6개의 토큰으로 분리됩니다.
토크나이저는 "이것이 NAME인가, STRING인가, OP인가"를 판별하는 첫 번째 관문입니다.
`SyntaxError` 가 발생하는 이유의 상당수는 이 단계에서 규칙을 어긴 것입니다.

연습문제

출력 예측 | 초급 | 문제 1

다음 코드의 출력 결과를 예측해보세요.

문제 코드

```
print("좋은 아침!")
print("오늘도 화이팅!")
```

정답 및 해설

출력:

```
좋은 아침!
오늘도 화이팅!
```

왜 이렇게 출력되는가?

`print()` 함수는 기본적으로 출력 후에 **줄바꿈**(`\n`)을 추가합니다.
함수 구조를 기억하세요: `print(*objects, end='\n')`
`end='\n'`이 기본값이기 때문에, 각 print마다 자동으로 줄이 바뀝니다.
만약 `print("좋은 아침!", end="")`로 쓰면 줄바꿈 없이 `좋은 아침!오늘도 화이팅!`이 됩니다.

출력 예측 중급 문제 2

다음 코드의 출력 결과를 예측해보세요. `sep`과 `end`의 역할을 생각해보세요.

문제 코드

```
print("사과", "바나나", "포도", sep="*")
print("안녕", "하세요", sep="", end="!\n")
print("끝")
```

정답 및 해설

출력:

```
사과*바나나*포도
안녕하세요!
끝
```

왜 이렇게 출력되는가?

1행: `sep="*"` 이므로 사과, 바나나, 포도 사이에 `*` 이 삽입됩니다. `end` 는 기본값 `'\n'` 이므로 줄바꿈.
2행: `sep=""` 이므로 "안녕"과 "하세요" 사이에 아무것도 없이 붙어서 `안녕하세요` . `end="!\n"` 이므로 끝에 `!` 과 줄바꿈이 붙어 `안녕하세요!`
3행: 모든 기본값 사용. 그냥 `끝` 출력.

핵심: `print` 의 동작은 함수 구조 `print(*objects, sep=' ', end='\n')` 를 알면 **모두 예측 가능**합니다. "왜 이렇게 되지?"가 아니라 "함수 정의가 이러니까 당연히 이렇다"가 됩니다.

직접 구현 초급 문제 3

`print` 를 사용해서 아래와 같은 자기소개 카드를 출력하세요.

원하는 출력:

```
====================
이름: 홍길동
나이: 15세
취미: 코딩
====================
```

> 힌트

`=` 기호를 20개 출력하려면 `print("=" * 20)` 을 사용할 수 있습니다.
파이썬에서 문자열에 `*` 를 쓰면 반복됩니다. (다음 주에 배울 내용 미리보기!)

> 정답 및 해설

정답 코드

```
print("=" * 20)
print("이름: 홍길동")
print("나이: 15세")
print("취미: 코딩")
print("=" * 20)
```

왜 이 코드가 답인가?

`"=" * 20`은 문자열 `"="`를 20번 반복합니다.
이것은 파이썬이 `*` 연산자를 문자열에도 적용할 수 있게 설계했기 때문입니다.
왜? 숫자의 곱셈이 "같은 수를 여러 번 더하는 것"이듯, 문자열의 곱셈은 "같은 문자를 여러 번 이어붙이는 것"으로 자연스럽게 확장한 것입니다.
이런 설계를 **연산자 오버로딩(operator overloading)**이라 합니다. (4주차에서 자세히 배웁니다)

직접 구현 | 중급 | 문제 4

`print` 함수를 **딱 한 번만** 사용해서 아래 출력을 만드세요.
(힌트: `sep`과 `end`를 활용하세요)

원하는 출력:

```
2026/03/30
```

정답 및 해설

정답 코드

```
print("2026", "03", "30", sep="/")
```

왜 이 코드가 답인가?

`print`에 여러 값을 넘기면, `sep` (separator, 구분자) 매개변수가 값들 사이에 자동으로 삽입됩니다.
기본값은 `sep=' '`(공백)이지만, `sep="/"`로 바꾸면 `/`가 들어갑니다.
이것이 함수의 **매개변수(parameter)**를 이해하는 첫 걸음입니다: 함수의 동작을 기본값이 아닌 다른 방식으로 바꿀 수 있습니다.

버그 찾기 중급 문제 5

아래 코드는 에러가 발생합니다. 무엇이 잘못되었고, 왜 잘못된 것인지 설명해보세요.

버그가 있는 코드

```
print("오늘의 날씨: 맑음)
print("기온: 15도")
```

정답 및 해설

에러 원인: 첫 번째 줄에서 닫는 따옴표 `"`가 빠졌습니다.

수정된 코드

```
print("오늘의 날씨: 맑음")
print("기온: 15도")
```

왜 이것이 에러인가?

파이썬은 `"`로 시작한 문자열은 반드시 `"`로 끝나야 한다고 기대합니다.
닫는 따옴표가 없으면, 파이썬은 "이 문자열이 어디서 끝나는지 모르겠다"며 `SyntaxError: unterminated string literal`을 발생시킵니다.

SyntaxError는 "문법 오류"라는 뜻입니다 — 파이썬의 문법 규칙을 어겼다는 의미입니다.
컴퓨터는 맥락을 추측하지 않습니다. 규칙에 정확히 맞아야만 이해합니다.

왜 이럴까? 고급 문제 6

아래 코드를 실행해보고, 결과가 왜 그렇게 나오는지 설명해보세요.

탐구 코드

```
print(print("안녕하세요"))
```

정답 및 해설

출력:

```
안녕하세요
None
```

왜 이렇게 출력되는가?

이 코드는 안에서 바깥으로 실행됩니다:

1. 먼저 안쪽 `print("안녕하세요")`가 실행됩니다 → 화면에 `안녕하세요` 출력
2. `print()` 함수는 화면에 출력만 하고, **돌려주는 값(return value)이 없습니다**
3. 돌려주는 값이 없는 함수는 자동으로 `None`을 반환합니다
4. 바깥쪽 `print(None)`이 실행됩니다 → 화면에 `None` 출력

`None`은 파이썬에서 **"아무것도 없음"**을 나타내는 특별한 값입니다.
핵심: `print()`는 화면에 "보여주는" 함수이지, 값을 "돌려주는" 함수가 아닙니다.
9주차에서 `return`과 함께 이 차이를 깊이 다룹니다.

AI 프롬프트 챌린지 중급 문제 7

AI(ChatGPT, Claude 등)에게 다음과 같이 요청해보세요:

> "print 함수만 사용해서 아래 모양을 출력하는 파이썬 코드를 만들어줘"

원하는 모양:

```
*
**
***
****
*****
```

AI가 만들어준 코드를 실행해보고, 다음 질문에 답해보세요:

1. AI가 만든 코드는 정상 작동하나요?
2. AI는 `for`문을 사용했나요? (우리는 아직 안 배웠습니다)
3. AI에게 **"for문 없이, print만 사용해서"**라고 다시 요청하면 어떤 코드가 나오나요?

해설

AI가 생성할 가능성이 높은 코드:

```
AI 답변 (for문 사용)

for i in range(1, 6):
    print("*" * i)
```

"for문 없이"라고 하면:

```
AI 답변 (print만 사용)

print("*")
print("**")
print("***")
print("****")
print("*****")
```

이 연습이 왜 중요한가?

AI는 보통 가장 "일반적인" 해법을 내놓습니다 (for문 사용).
하지만 우리가 원하는 제약 조건("for문 없이")을 명시하면, 다른 방법을 찾아줍니다.
AI에게 정확한 지시를 내리는 능력이 바이브 코딩 시대의 핵심 역량입니다.
그리고 AI가 `"*" * i`를 쓴다면, 이것이 문자열 반복이라는 걸 **이해**해야 검증할 수 있습니다.

AST 분석 | 전공자 | 문제 8

아래 두 코드를 `ast.parse()` + `ast.dump()`로 분석하세요.
AST 출력의 어떤 부분이 다른가요? 그리고 왜 다른가요?

```
비교할 두 코드

import ast

code_a = 'print("Hello")'
code_b = 'print("A", "B", sep="-")'
```

```
tree_a = ast.parse(code_a)
tree_b = ast.parse(code_b)

print("=== Code A ===")
print(ast.dump(tree_a, indent=2))
print("\n=== Code B ===")
print(ast.dump(tree_b, indent=2))
```

정답 및 해설

CODE A의 핵심 부분:

```
Call(
  func=Name(id='print'),
  args=[Constant(value='Hello')],
  keywords=[]
)
```

CODE B의 핵심 부분:

```
Call(
  func=Name(id='print'),
  args=[Constant(value='A'), Constant(value='B')],
  keywords=[
    keyword(arg='sep', value=Constant(value='-'))
  ]
)
```

왜 이렇게 다른가?

AST의 `Call` 노드는 **위치 인자(args)**와 **키워드 인자(keywords)**를 분리하여 저장합니다.
`sep="-"`처럼 이름이 있는 인자는 `keywords` 리스트에 들어가고, `keyword` 객체는 `arg`(이름)와 `value`(값)를 가집니다.

이것이 Python 인터프리터가 `print("A", "B", sep="-")`를 파싱할 때 `sep`을 "두 번째 문자열 인자"가 아닌 "키워드 인자"로 처리할 수 있는 이유입니다.
AST 레벨에서 이미 구분이 일어납니다.

바이트코드 비교 | 전공자 | 문제 9

아래 두 함수의 바이트코드를 `dis.dis()`로 비교하세요. 어떤 명령어가 추가되었고, 왜 추가되었는가요?

비교 코드

```
import dis

def func_a():
    print("Hello")        # 반환값을 버림

def func_b():
    x = print("Hello")    # 반환값을 변수에 저장

print("=== func_a ===")
dis.dis(func_a)
print("\n=== func_b ===")
dis.dis(func_b)
```

정답 및 해설

```
FUNC_A 바이트코드:

  PUSH_NULL
  LOAD_GLOBAL     print
  LOAD_CONST      'Hello'
  CALL            1
  POP_TOP           ← print()의 None 반환값을 스택에서 버림
  RETURN_CONST    None
```

```
FUNC_B 바이트코드:

  PUSH_NULL
  LOAD_GLOBAL     print
  LOAD_CONST      'Hello'
  CALL            1
                    ← POP_TOP 없음! 대신:
  STORE_FAST      x  ← 반환값을 x에 저장 (함수 내 로컬 변수는 STORE_FAST)
  RETURN_CONST    None
```

왜 다른가?

`func_a`: `print()` 호출 후 반환값(`None`)이 스택에 쌓이는데, 아무도 그것을 쓰지 않습니다. 컴파일러는 이것을 감지하고 `POP_TOP`(스택 최상단 버리기)을 삽입합니다.

`func_b`: 반환값을 `x`에 저장하므로 `POP_TOP` 대신 `STORE_FAST x`가 옵니다. (`STORE_NAME`은 모듈 레벨, 함수 내부 로컬 변수는 `STORE_FAST`)

핵심: `x = print("Hello")`를 실행하면 `x`에는 `None`이 저장됩니다. `print()`의 **부수 효과(side effect)**는 화면 출력이고, **반환값(return value)**은 `None`입니다. 이 두 가지를 구분하는 것이 9주차 함수 이해의 핵심입니다.

파이프라인 추적 | 전공자 | 문제 10

아래 각 코드에서 에러가 발생합니다. 파이썬 컴파일 파이프라인(Tokenizer → Parser → AST → Compiler → PVM)의 **어느 단계에서 각 에러가 감지되는지** 맞춰보세요.

에러 코드 4개

```
# 코드 A
print("Hello          # 닫는 따옴표 없음

# 코드 B
print("Hello")
  print("World")      # 잘못된 들여쓰기

# 코드 C
x = 1 +               # 불완전한 표현식

# 코드 D
print(존재하지않는변수)  # 실행 시 이름 없음
```

정답 및 해설

코드	에러	감지 단계	이유
A	`SyntaxError: unterminated string literal`	**Tokenizer**	문자열 토큰을 만들다가 닫는 따옴표 없이 줄 끝 도달 — 토큰 자체를 완성 못함
B	`IndentationError: unexpected indent`	**Parser**	들여쓰기는 Python 문법(INDENT/DEDENT 토큰)의 일부 — 파서가 문법 규칙 위반 감지
C	`SyntaxError: invalid syntax`	**Parser**	`+` 뒤에 피연산자가 없음 — 문법 규칙(expression → term op term)의 `term`이 없음
D	`NameError: name '존재하지않는변수' is not defined`	**PVM (실행 시)**	문법적으로는 완전히 유효한 코드 — `LOAD_GLOBAL` 명령어가 실행될 때 네임스페이스에서 이름을 찾지 못함

왜 이 구분이 중요한가?

SyntaxError / IndentationError: 실행 전에 감지됩니다. 파이썬은 코드 전체를 먼저 파싱한 후 실행하기 때문에, 파일 끝에 있는 문법 에러도 첫 줄 실행 전에 발견됩니다.

NameError / TypeError / ValueError: 실행 중에 감지됩니다. 해당 코드가 실제로 실행(PVM이 그 바이트코드에 도달)될 때만 에러가 납니다.

LLM과의 연결: LLM이 생성한 코드에서 SyntaxError는 비교적 쉽게 잡을 수 있지만, NameError · TypeError는 실제 실행 없이는 잡기 어렵습니다. 이것이 "실행 피드백(execution feedback)"이 LLM 코드 품질 개선에 중요한 이유입니다.

이번 주 정리

- **컴퓨터**는 0과 1만 이해하는 아주 빠른 계산기입니다
- **프로그래밍 언어**는 사람의 의도를 0과 1로 번역하는 통역사입니다
- **파이썬**은 가장 사람의 말에 가까운 인터프리터 언어입니다
- **REPL**(`>>>`)은 파이썬과 한 줄씩 대화하는 인터랙티브 모드입니다
- `#`으로 시작하는 줄은 **주석**으로, 파이썬이 무시합니다
- `print()`는 단순한 명령어가 아니라 **함수(function)**이며, 파이썬 2의 `print "Hello"`와 달리 파이썬 3에서는 반드시 괄호를 씁니다
- 파이썬에서 **모든 것은 객체(object)**입니다 — print도 객체입니다
- `help()`와 `dir()`로 파이썬 안에서 파이썬을 배울 수 있습니다
- **AI**는 패턴 매칭을 하므로, 코딩의 원리를 이해해야 AI의 실수를 검증할 수 있습니다
- [전공자] **CPython**은 레퍼런스 구현체 — 언어 명세(Python Language Reference)와 구현체는 다릅니다
- [전공자] `print()`는 `sys.stdout.write()`를 호출하며, FD 1을 통해 커널 → 터미널로 전달됩니다
- [전공자] 파이썬 컴파일 파이프라인: 소스 → 토크나이저 → 파서 → AST → 바이트코드 → PVM

- [전공자] **GIL**은 참조 카운팅 Thread-Safety를 위해 존재하며, 진짜 병렬 CPU 실행을 제약합니다

이번 주 용어

CPU

Central Processing Unit. 컴퓨터의 두뇌. 계산을 실행하는 부품

RAM (메모리)

Random Access Memory. 작업 중인 데이터를 임시 저장하는 공간

Interpreter

인터프리터. 소스 코드를 읽어 실행하는 프로그램. CPython은 내부적으로 바이트코드로 컴파일한 뒤 PVM에서 실행

Compiler

컴파일러. 코드 전체를 기계어로 변환한 후 실행하는 프로그램

Function (함수)

특정 작업을 수행하는 코드 묶음. `print()`가 대표적 예시

Parameter (매개변수)

함수에 전달하는 설정값. `sep`, `end` 등

Argument (인자)

함수를 호출할 때 실제로 전달하는 값. `print("안녕")`에서 `"안녕"`이 인자

Return Value (반환값)

함수가 실행 후 돌려주는 값. `print()`는 `None`을 반환

Object (객체)

파이썬에서 모든 데이터의 기본 단위. 숫자, 문자열, 함수 모두 객체

Bytecode (바이트코드)

파이썬 소스 코드를 PVM이 실행할 수 있게 변환한 중간 코드

PVM (Python Virtual Machine)

파이썬 가상 머신. 바이트코드를 스택 기반으로 한 줄씩 실행하는 엔진

Vibe Coding

AI에게 자연어로 지시하여 코드를 생성하는 프로그래밍 방식

None

파이썬에서 "아무 값도 없음"을 나타내는 특별한 객체

REPL

Read-Eval-Print Loop. `>>>` 프롬프트로 파이썬과 한 줄씩 대화하는 인터랙티브 모드

Comment (주석)

`#`으로 시작하는 줄. 파이썬이 무시하며, 사람을 위한 설명을 씁니다

Hallucination (할루시네이션)

AI가 그럴듯해 보이지만 틀린 코드를 생성하는 현상. 패턴 매칭의 한계

Script Mode (스크립트 모드)

`.py` 파일로 저장 후 `python 파일명.py`로 실행하는 방식

dis 모듈

Python 코드의 바이트코드를 사람이 읽을 수 있게 보여주는 내장 모듈

SyntaxError

파이썬의 문법 규칙을 어겼을 때 발생하는 오류. 실행 전 인터프리터가 감지

CPython

C언어로 구현된 파이썬 레퍼런스 구현체. python.org에서 다운받는 파이썬이 CPython

GIL (Global Interpreter Lock)

CPython의 전역 잠금. 참조 카운팅 안전성을 위해 한 번에 하나의 스레드만 바이트코드 실행

PEP (Python Enhancement Proposal)

파이썬 언어 변경 제안 공식 문서. print 함수화는 PEP 3105, walrus 연산자는 PEP 572

AST (Abstract Syntax Tree)

소스 코드를 구문 분석한 추상 트리 구조. `ast` 모듈로 직접 확인 가능

sys.stdout

파이썬의 표준 출력 스트림 객체. print()는 내부적으로 이것의 write()를 호출

File Descriptor (파일 디스크립터)

OS가 파일/스트림에 부여하는 정수 식별자. 0=stdin, 1=stdout, 2=stderr

Reference Counting (참조 카운팅)

CPython의 메모리 관리 방식. 객체를 가리키는 참조 수가 0이 되면 즉시 메모리 해제

__pycache__ / .pyc

컴파일된 바이트코드 캐시 파일. 다음 실행 때 재컴파일 생략을 위해 저장

다음 주 맛보기

2주차: 데이터와 타입 — 컴퓨터는 숫자와 글자를 어떻게 구분하는가?

`42` 와 `"42"` 는 같을까요, 다를까요?
컴퓨터는 숫자 42와 문자열 "42"를 완전히 다르게 취급합니다.

> 왜 `int`, `str`, `float`, `bool`이라는 타입이 필요한지,
> `type()` 함수로 모든 것의 정체를 밝혀봅니다.

참고문헌

본 챕터의 기술적 주장은 모두 아래 1차 출처와 대조하여 검증되었습니다. URL은 모두 2026년 4월 30일 마지막 접속 기준입니다.

[1] **Python Software Foundation. Python 3.14 Tutorial — Chapter 1: Whetting Your Appetite.** docs.python.org/3/tutorial/appetite.html (한국어판: `tutorial/appetite.txt`)

> "파이썬은 인터프리터 언어입니다. 컴파일과 링크 단계가 필요 없으므로 개발 시간을 상당히 단축해줍니다."(§1 ¶42–43)

[2] **The Unicode Consortium. The Unicode Standard, Version 16.0.** unicode.org/versions/Unicode16.0.0/ — Code point U+0041 (LATIN CAPITAL LETTER A) = 십진수 65 = 이진수 1000001. ASCII 표준(ANSI X3.4-1986 / RFC 20)과 호환.

[3] **Barry Warsaw. PEP 3147 — PYC Repository Directories. Python.org, 2009.** peps.python.org/pep-3147/ — Python 3.2부터 컴파일된 바이트코드를 `__pycache__/<module>.cpython-<version>.pyc` 형식으로 캐시.

[4] **Guido van Rossum. "A Brief Timeline of Python." The History of Python (blog), 2009.** python-history.blogspot.com — Python 구현은 1989년 12월 (CWI 크리스마스 휴가 기간)에 시작되어 1991년 2월 첫 공개.

[5] **Tim Peters. PEP 20 — The Zen of Python. Python.org, 2004.** peps.python.org/pep-0020/ — REPL에서 `import this`로 확인 가능. 19가지 설계 원칙.

[6] **PyPy Project. PyPy Speed Center.** speed.pypy.org — 공식 벤치마크 스위트의 기하평균 비교. 워크로드별로 큰 편차가 존재하며 수치는 시점·플랫폼에 따라 변동.

[7] **Python Software Foundation. Python Enhancement Proposals (PEPs).** peps.python.org — 모든 PEP의 공식 발행 사이트. 소스 저장소: `github.com/python/peps`.

[8] **Python Software Foundation. Python 3.14 Library Reference — Built-in Functions: `print()`.** docs.python.org/3/library/functions.html#print (한국어판: `library/functions.txt` §1465–1480) — 시그니처: `print(*objects, sep=' ', end='\n', file=None, flush=False)`. "file 인자가 존재하지 않거나 None이면, sys.stdout이 사용됩니다."

[9] **Georg Brandl. PEP 3105 — Make print a function. Python.org, 2006.** peps.python.org/pep-3105/ — Python 3에서 `print`를 statement에서 builtin 함수로 변경. **저자: Georg Brandl** (Guido van Rossum이 아님).

[10] **Guido van Rossum, Pablo Galindo, Lysandros Nikolaou. PEP 617 — New PEG parser for CPython. Python.org, 2020.** peps.python.org/pep-0617/ — Python 3.9부터 LL(1) 파서를 PEG(Parsing Expression Grammar)로 교체. 3.10에서 구 파서 완전 제거.

[11] **Python Software Foundation. Python 3.14 Tutorial — Chapter 2.1.2: Interactive Mode.** docs.python.org/3/tutorial/interpreter.html#interactive-mode (한국어판: `tutorial/interpreter.txt` § 77–80) — "명령을 tty 에서 읽을 때, 인터프리터가 대화형 모드로 동작한다고 말합니다."

PYWHY · 무료 미리보기

여기까지가 미리보기입니다

이 책은 총 **15주차 998쪽**으로
1주차에서 시작해 동시성·비동기까지 이어집니다.

for문이 iter()/next()로 풀리고,
클래스가 __init__/__new__/__class__로 분해되며,
asyncio가 GIL과 어떻게 공존하는지
인터프리터의 시선으로 끝까지 따라갑니다.

전체 책 구매 · 모든 주차 + 색인 + 부록

→ **soccz.github.io/python-why**

H0NG · PyWhy v1.0